

Aula prática

API de Tarefas com Node.js e MySQL 8.0

Preparando o ambiente

1. Instale o Node.js LTS.
2. Instale o MySQL 8.0 e configure um usuário (ex.: root com senha senha123).
3. Crie uma pasta para o projeto:

```
mkdir api-tarefas  
cd api-tarefas
```

4. Inicialize o projeto:

```
npm init -y
```

Instalando dependências

```
npm install express mysql2 bcrypt jsonwebtoken
```

express → framework web.

mysql2 → driver para MySQL.

bcrypt → criptografia de senhas.

jsonwebtoken → autenticação via token JWT.

Criando o banco e tabelas

No MySQL, execute:

```
CREATE DATABASE api_tarefas;  
USE api_tarefas;
```

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  username VARCHAR(50) NOT NULL UNIQUE,  
  passwordHash VARCHAR(255) NOT NULL  
);
```

```
CREATE TABLE tasks (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  title VARCHAR(100),  
  description TEXT,  
  done BOOLEAN DEFAULT FALSE,  
  userId INT,
```

```
    FOREIGN KEY (userId) REFERENCES users(id)
  );
```

Estrutura de arquivos

```
api-tarefas/
├── server.js
├── db.js
├── routes.js
└── auth.js
```

Conexão com MySQL (db.js)

```
const mysql = require('mysql2');

const db = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: 'senha123', // ajuste conforme sua instalação
  database: 'api_tarefas'
});

module.exports = db;
```

Autenticação (auth.js)

```
const jwt = require('jsonwebtoken');
const SECRET = 'segredo123'; // em produção, use variável de ambiente

function generateToken(user) {
  return jwt.sign({ id: user.id }, SECRET, { expiresIn: '1h' });
}

function verifyToken(req, res, next) {
  const token = req.headers['authorization'];
  if (!token) return res.status(401).send("Token não fornecido");

  jwt.verify(token, SECRET, (err, decoded) => {
    if (err) return res.status(401).send("Token inválido");
    req.userId = decoded.id;
    next();
  });
}

module.exports = { generateToken, verifyToken };
```

Rotas (routes.js)

```
const express = require('express');
const bcrypt = require('bcrypt');
const db = require('./db');
const { generateToken, verifyToken } = require('./auth');

const router = express.Router();

// Registro de usuário
router.post('/register', (req, res) => {
  const { username, password } = req.body;
  const passwordHash = bcrypt.hashSync(password, 10);

  db.query("INSERT INTO users (username, passwordHash) VALUES (?, ?)",
    [username, passwordHash],
    (err, result) => {
      if (err) return res.status(500).send(err.message);
      res.send("Usuário registrado com sucesso!");
    });
});

// Login
router.post('/login', (req, res) => {
  const { username, password } = req.body;
  db.query("SELECT * FROM users WHERE username = ?", [username], (err, results) => {
    if (err) return res.status(500).send(err.message);
    const user = results[0];
    if (!user) return res.status(404).send("Usuário não encontrado");
    if (!bcrypt.compareSync(password, user.passwordHash)) return
    res.status(401).send("Senha inválida");

    const token = generateToken(user);
    res.json({ token });
  });
});

// Listar tarefas
router.get('/tasks', verifyToken, (req, res) => {
  db.query("SELECT * FROM tasks WHERE userId = ?", [req.userId], (err, rows) => {
    if (err) return res.status(500).send(err.message);
    res.json(rows);
  });
});

// Criar tarefa
router.post('/tasks', verifyToken, (req, res) => {
  const { title, description } = req.body;
  db.query("INSERT INTO tasks (title, description, done, userId) VALUES (?, ?, ?, ?)",
    [title, description, false, req.userId],
    (err, result) => {
      if (err) return res.status(500).send(err.message);
      res.send("Tarefa criada!");
    });
});

// Atualizar tarefa
router.put('/tasks/:id', verifyToken, (req, res) => {
```

```

const { title, description, done } = req.body;
db.query("UPDATE tasks SET title=?, description=?, done=? WHERE id=? AND userId=?",
  [title, description, done, req.params.id, req.userId],
  (err, result) => {
    if (err) return res.status(500).send(err.message);
    res.send("Tarefa atualizada!");
  });
});

// Deletar tarefa
router.delete('/tasks/:id', verifyToken, (req, res) => {
  db.query("DELETE FROM tasks WHERE id=? AND userId=?", [req.params.id, req.userId], (err,
  result) => {
    if (err) return res.status(500).send(err.message);
    res.send("Tarefa removida!");
  });
});

module.exports = router;

```

Servidor principal (server.js)

```

const express = require('express');
const routes = require('./routes');

const app = express();
app.use(express.json());

app.use(routes);

app.listen(3000, () => {
  console.log('Servidor rodando em http://localhost:3000');
});

```

Testando a API

1. Inicie o servidor:

```
node server.js
```

2. Teste com Postman/Insomnia:

POST /register → criar usuário.

POST /login → obter token.

Usar token no header Authorization para acessar rotas /tasks.

Nesta atividade praticamos:

- Configuração de servidor Node.js com Express.
- Conexão e manipulação de dados no MySQL 8.0.
- Autenticação com JWT.
- Criptografia de senhas com bcrypt.
- CRUD completo com banco relacional MySQL.